

day5-2 源码与架构说明

18 个文件 · 979 行 · 生成于 2026-10-06 09:42 · 行号对应源文件实际行号

README.md

Markdown

day5-2: C++ 业务核心 + HTML 界面

界面用 HTML/CSS/JS 写，业务逻辑用 C++ 写，两边通过 webview 的函数绑定通信。窗口和网页渲染交给系统 webview，不依赖 Qt。

这个项目同时是一份**最小架构的参考实例**：目录分层、依赖方向、错误处理策略 都已经定好，可以直接照着开下一个项目。

依赖

- CMake ≥ 3.16，以及支持 C++17 的编译器
- GTK 3 + WebKitGTK 4.1 (pkg-config 能找到 gtk+-3.0 与 webkit2gtk-4.1)
- 第三方头文件已随项目内置，不需要额外安装：third_party/webview/webview.h (webview 0.12.0)、third_party/nlohmann/json.hpp (nlohmann/json 3.12.0)

目录结构

```
day5-2/
├── CMakeLists.txt                # 顶层：C++ 标准、输出目录、第三方目标、三个子目录
├── core/                        # 业务核心：纯 C++，零 UI、零 IO
│   ├── CMakeLists.txt          # -> 静态库 day5_core
│   ├── include/day5_2/         # 对外头文件（外部只能看到这里）
│   │   ├── result.h            # ErrorCode / Error / Result<T>
│   │   └── person.h            # Person 记录 + PersonService 业务逻辑
│   └── src/
│       ├── person.cpp
│       └── text_utils.h         # 模块内部工具，不对外暴露
├── app/                         # 界面外壳：UI + 组装
│   ├── CMakeLists.txt          # -> 可执行文件 day5_2
│   ├── main.cpp                # 组装根：把 core 和界面接起来
│   ├── controller.h/.cpp       # 控制器：界面动作 <-> 业务调用
│   ├── presenter.h/.cpp        # 展示层：全部中文文案在这里
│   ├── protocol.h/.cpp         # 协议：JSON 编解码、信封定义
│   └── web/index.html           # 视图（HTML + CSS + JS 单文件）
├── tests/                       # 脱离界面的业务测试
│   ├── CMakeLists.txt
│   └── person_test.cpp
└── third_party/                 # 内置依赖
    ├── webview/webview.h
    └── nlohmann/json.hpp
```

四条规则

这四条是整个架构的**本体**，目录只是它们的表现形式。

规则一：依赖只朝一个方向。 app → core，永远不反向。core 不 include webview、不碰界面、不写文件；app 负责把业务结果变成界面能用的东西。唯一的例外是 main.cpp，它是组装根，可以同时看到两边。

这条不靠自觉，靠构建系统保证：day5_core 的链接列表里没有 GTK、没有 webview，所以谁在 core 里 include 界面头文件，**编译会直接失败**。

规则二：一个目录一种角色。

目录	只放	绝对不放
core/	业务规则、数据结构	界面代码、JSON、日志输出
app/	界面、渲染、组装	业务规则、校验逻辑
tests/	测试	被产品代码包含的东西

配套命名法：文件名 = 主类型名，全小写下划线；命名空间 = day5_2； include 一律从项目根写全路径（#include "day5_2/person.h"），不用 ./ 或 ../。

规则三：边界只交换两种东西。进去的是类型化的参数，出来的是 Result<T>。core 从不产出给用户看的句子，它返回机器可读的失败标识（如 person.name.empty），中文案由 app/presenter.cpp 一处负责。换语言、换图标只改那一个文件。

规则四：错误处理策略第一天就定死。

- 可预期的失败（用户没填姓名、年龄越界）→ 返回值 Result<T>
- 不可恢复的失败（构造不出对象、文件打不开）→ 抛异常

另有一条硬约束：异常绝对不许越过 C 接口边界。webview 的绑定回调是 C 接口，从里面抛异常是未定义行为，所以这一层的解析一律用 json::parse(args, nullptr, false) 这种不抛异常的写法。

线上协议

页面加载后，webview 把三个函数注入到 JS 的 window 上，调用后返回 Promise：

```
window.getState()           // 查询状态
window.submitEntry(name, age) // 提交一条记录
window.clearMessages()       // 清空
```

返回的 JSON 信封（字段名就是前后端契约）：

```
{
  "ok": false,
  "level": "error",
  "event": "failed",
  "status": "x 年龄必须在 1-150 之间",
  "detail": "年龄必须在 1-150 之间",
  "error": "person.age.out_of_range",
  "messages": ["张三 (20 岁)"]
}
```

字段	含义
ok	本次操作是否成功
level	info 中性 / ok 成功 / error 失败，界面据此决定样式
event	state / submitted / failed / cleared
status	状态栏文本，可以直接显示
detail	本条记录的摘要，或失败说明
error	失败时机器可读的原因；成功为空
messages	列表内容（已格式化）

界面只认 ok 和 level，不去嗅探 **❌ 这类文本前缀**——文案可以在后端随便改，界面不受影响。需要精确处理某个错误时，用 error 字段判分支。

加一个新业务，要动哪些文件

以"新增一个订单模块"为例，固定就这几步：

1. core/include/day5_2/<业务>.h — 数据结构 + 业务类声明
2. core/src/<业务>.cpp — 业务规则，失败返回 Result + Error{code, key}
3. core/CMakeLists.txt 里把新 .cpp 加进 day5_core
4. tests/<业务>_test.cpp + tests/CMakeLists.txt — 先测业务，再接界面
5. app/presenter.cpp 的 lookup 里给新 key 加中文（多数情况复用已有错误码，不用改）
6. app/controller.cpp 里注册一行绑定

第 5、6 步通常只有几行。判断这个架构是否好用的标准就是：**加业务不需要改架构代码。**

构建与运行

```
cmake -S . -B build -G Ninja
cmake --build build
./build/bin/day5_2          # 需要图形环境
```

跑测试（不需要图形环境）：

```
cd build && ctest --output-on-failure
```

若干说明

- **依赖方向可验证**: day5_core 只暴露 core/include/, 链接列表里没有任何界面库。 想确认架构没被破坏, 看一眼 core/CMakeLists.txt 就够了。
- **头文件路径**: 公开头文件放在 include/day5_2/ 下, 按 #include "day5_2/xxx.h" 引用; 只在本模块内部使用的头文件 (如 src/text_utils.h) 留在 src/ 里, 不进 include/。 这样"哪个文件是给别人用的"一目了然。
- **Result<T> 不是自创**: C++23 已经把它标准化为 std::expected<T, E>, 等编译器支持了直接替换, 业务代码不用改。
- **JSON 编解码用 nlohmann/json**: 顶层 CMakeLists 把它包成 INTERFACE 目标 nlohmann_json::nlohmann_json, 用法与官方 find_package 一致, 以后换系统包时链接处不用改。
- **回包用 ordered_json**: 默认的 json 底层是 std::map, dump() 会按 key 字母序输出; ordered_json 保持写入顺序, 读日志更直观。
- **HTML 在构建期内嵌**: CMake 把 app/web/index.html 转成 index_html.h, 运行时不需要额外文件, 改完 HTML 要重新构建。 想边改边看, 把 main.cpp 里的 window.set_html(kIndexHtml) 换成 window.navigate("file:///绝对路径/app/web/index.html")。
- **index.html 里不要写 CMake 的 "\${...}" 形式**, 否则会在配置阶段被展开。
- 当前使用 webview 0.12.0 (webview::webview 是 browser_engine 的别名) + GTK 3 + WebKitGTK 4.1。

什么时候需要加目录

目录随需要增长, 不提前建空目录。等哪天遇到下面这些情况再动:

触发条件	加什么
出现第二种界面 (比如同时要 Qt 版)	app/<壳名>/ 拆目录, presenter/protocol 提为共用
core 里出现第二个业务域	core/include/day5_2/<域>/ 分子目录
存储要换实现 (内存 → SQLite)	抽 <域>_repository.h 接口, 实现放 app/
出现第二套传输 (REST/CLI)	抽 presenter/protocol 的公共部分

```
1 cmake_minimum_required(VERSION 3.16)
2 project(day5_2 LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 17)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6 set(CMAKE_CXX_EXTENSIONS OFF)
7 set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
8 set(CMAKE_RUNTIME_OUTPUT_DIRECTORY "${CMAKE_BINARY_DIR}/bin")
9
10 # 第三方单头文件 (webview、nlohmann/json) 所在目录，谁需要谁引用。
11 set(DAY5_THIRD_PARTY_DIR "${CMAKE_CURRENT_SOURCE_DIR}/third_party")
12
13 # nlohmann/json: third_party/nlohmann/json.hpp 里的单头文件。
14 # 包成 INTERFACE 目标，并对齐官方包提供的目标名 nlohmann_json::nlohmann_json，
15 # 这样以后想换成 find_package(nlohmann_json REQUIRED)，各外壳的链接处一行都不用改。
16 add_library(nlohmann_json_vendored INTERFACE)
17 target_include_directories(nlohmann_json_vendored INTERFACE "${DAY5_THIRD_PARTY_DIR}")
18 add_library(nlohmann_json::nlohmann_json ALIAS nlohmann_json_vendored)
19
20 # 分层: core = 纯 C++ 业务逻辑; app = 界面外壳; tests = 脱离界面的业务测试。
21 # 依赖方向只有一条: app -> core。core 的目标里没有任何界面库，
22 # 所以“业务层不许碰界面”这条规则由构建系统强制，不靠自觉。
23 enable_testing()
24 add_subdirectory(core)
25 add_subdirectory(app)
26 add_subdirectory(tests)
```

```
1  # day5_core: 纯 C++ 业务逻辑 (core/ 层)。
2  #
3  # 规则一：依赖只朝一个方向。这个目标的链接列表里没有 GTK、没有 webview，
4  # 所以谁在 core 里 include 界面头文件，编译会直接失败—靠构建系统守规矩。
5  # 规则二：一个目录一种角色。core/ 只放业务，界面代码一律放 app/。
6  # 规则三：公开头文件在 include/<项目名>/ 下，外部按 "day5_2/xxx.h" 引用。
7  add_library(day5_core STATIC
8      src/person.cpp
9  )
10
11  target_include_directories(day5_core PUBLIC
12      "${CMAKE_CURRENT_SOURCE_DIR}/include"
13  )
14
15  # 规则四：错误处理策略和告警级别第一天就定死，不靠事后 review。
16  target_compile_options(day5_core PRIVATE
17      -Wall -Wextra -Wpedantic
18  )
```

```
1  #pragma once
2
3  #include <string>
4
5  namespace day5_2 {
6
7  // 业务失败的分类。用 enum class 而不是字符串：写错名字会编译失败。
8  enum class ErrorCode {
9      none,
10     invalid_input, // 输入不合法：可预期的失败，用返回值报告
11     not_found,
12     conflict,
13     internal,
14 };
15
16 // 一次业务失败的描述。
17 //   code   : 失败分类，界面可以据此做粗粒度的统一处理
18 //   key     : 机器可读的稳定标识，界面可以据此精确反应或做多语言
19 //   detail  : 补充数据（比如出错的原始数值），不是给用户看的句子
20 struct Error {
21     ErrorCode code = ErrorCode::none;
22     std::string key;
23     std::string detail;
24
25     [[nodiscard]] bool ok() const { return code == ErrorCode::none; }
26 };
27
28 // 业务函数的统一返回形状：要么拿到值，要么拿到失败原因。
29 // C++23 把等价的类型标准化成了 std::expected<T, E>，届时可以直接替换掉它。
30 template <typename T>
31 struct Result {
32     T value{};
33     Error error{};
34
35     [[nodiscard]] bool ok() const { return error.ok(); }
36 };
37
38 } // namespace day5_2
```

```
1  #pragma once
2
3  #include "day5_2/result.h"
4
5  #include <string>
6  #include <string_view>
7  #include <vector>
8
9  namespace day5_2 {
10
11  // 一条记录的业务数据。这里只存事实，不存任何展示用的文本。
12  struct Person {
13      std::string name;
14      int age = 0;
15  };
16
17  // 人员列表的业务逻辑：校验、保存、清空。
18  // 它不依赖界面，也不生成任何给用户看的文本。
19  class PersonService {
20  public:
21      // 校验并保存。失败时值不可用，改为读 error。
22      Result<Person> submit(std::string_view name, int age);
23
24      void clear();
25
26      const std::vector<Person> &people() const { return m_people; }
27
28  private:
29      std::vector<Person> m_people;
30  };
31
32 } // namespace day5_2
```

```
1  #pragma once
2
3  #include <cctype>
4  #include <string>
5  #include <string_view>
6
7  // 模块内部工具：放在 src/ 下，不对外暴露（不会进 include/）。
8  namespace day5_2::detail {
9
10     inline std::string trim(std::string_view text) {
11         std::size_t begin = 0;
12         std::size_t end = text.size();
13         while (begin < end && std::isspace(static_cast<unsigned char>(text[begin])))
14             ++begin;
15         while (end > begin && std::isspace(static_cast<unsigned char>(text[end - 1])))
16             --end;
17         return std::string{text.substr(begin, end - begin)};
18     }
19
20 } // namespace day5_2::detail
```

```
1  #include "day5_2/person.h"
2
3  #include "text_utils.h"
4
5  #include <string>
6
7  namespace day5_2 {
8
9  namespace {
10
11     constexpr int kMinAge = 1;
12     constexpr int kMaxAge = 150;
13
14 } // namespace
15
16 Result<Person> PersonService::submit(std::string_view name, int age) {
17     const std::string trimmed = detail::trim(name);
18     if (trimmed.empty())
19         return {{}, {ErrorCode::invalid_input, "person.name.empty", {}}};
20
21     if (age < kMinAge || age > kMaxAge)
22         return {{}, {ErrorCode::invalid_input, "person.age.out_of_range", std::to_string(age)}};
23
24     m_people.push_back(Person{trimmed, age});
25     return {m_people.back(), {}};
26 }
27
28 void PersonService::clear() {
29     m_people.clear();
30 }
31
32 } // namespace day5_2
```

```
1 # app: 界面外壳 (可执行目标 day5_2)。
2 #
3 # 它是唯一允许同时看到 core 和界面库的地方:
4 # 规则—在这里体现为 target_link_libraries — core 不被允许反向依赖 app。
5
6 # ---- 依赖: GTK 3 + WebKitGTK (webview 在 Linux 上的后端) ----
7 find_package(PkgConfig REQUIRED)
8 pkg_check_modules(GTK REQUIRED IMPORTED_TARGET gtk+-3.0)
9 pkg_check_modules(WEBKITGTK REQUIRED IMPORTED_TARGET webkit2gtk-4.1)
10
11 # ---- 把 web/index.html 内嵌成 C++ 头文件 ----
12 # 注意: index.html 里不要出现 CMake 的 “${...}” 写法, 否则会在配置阶段被展开。
13 set(WEB_DIR "${CMAKE_CURRENT_SOURCE_DIR}/web")
14 set(GENERATED_DIR "${CMAKE_CURRENT_BINARY_DIR}/generated")
15 set(GENERATED_HEADER "${GENERATED_DIR}/index_html.h")
16
17 file(READ "${WEB_DIR}/index.html" INDEX_HTML)
18 file(MAKE_DIRECTORY "${GENERATED_DIR}")
19 file(WRITE "${GENERATED_HEADER}"
20     "// 该文件由 CMake 从 app/web/index.html 生成, 请勿手动修改。\\n"
21     "static const char *kIndexHtml = R\\\"WEBVIEW_HTML(\\n${INDEX_HTML})WEBVIEW_HTML\\\";\\n"
22 )
23 set_property(DIRECTORY APPEND PROPERTY CMAKE_CONFIGURE_DEPENDS "${WEB_DIR}/index.html")
24
25 # ---- 可执行程序 ----
26 # 职责切分: main = 组装, controller = 翻译, protocol = 编解码, presenter = 文案。
27 add_executable(day5_2
28     main.cpp
29     controller.cpp
30     protocol.cpp
31     presenter.cpp
32 )
33
34 target_include_directories(day5_2 PRIVATE
35     "${CMAKE_CURRENT_SOURCE_DIR}" # controller.h / protocol.h / presenter.h
36     "${GENERATED_DIR}"           # 生成的 index_html.h
37     "${DAY5_THIRD_PARTY_DIR}"     # webview/webview.h
38 )
39
40 target_link_libraries(day5_2 PRIVATE
41     day5_core
42     nlohmann_json::nlohmann_json
43     PkgConfig::GTK
44     PkgConfig::WEBKITGTK
45     ${CMAKE_DL_LIBS}
46 )
47
48 target_compile_options(day5_2 PRIVATE
49     -Wall -Wextra -Wpedantic
50 )
```

```
1 // 组装根：只负责把 core（业务）和 app（界面）接起来，不写业务、不写文案。
2 #include "controller.h"
3 #include "presenter.h"
4
5 #include "day5_2/person.h"
6
7 #include <webview/webview.h>
8
9 #include "index_html.h" // 由 CMake 从 web/index.html 生成
10
11 #include <iostream>
12
13 int main() {
14     try {
15         // 第一个参数 true 会打开开发者工具；第二个参数传 nullptr 表示自己建窗口。
16         webview::webview window(false, nullptr);
17         window.set_title("Mini 数据流转示例 (webview + HTML + C++)");
18         window.set_size(420, 560, WEBVIEW_HINT_NONE);
19
20         day5_2::PersonService service; // 业务：数据与规则
21         day5_2::app::Presenter presenter; // 展示：所有中文文案
22         day5_2::app::Controller controller(window, service, presenter); // 控制器：翻译
23         controller.bind();
24
25         // 页面在编译期内嵌进可执行文件。想边改边看，换成：
26         // window.navigate("file:///绝对路径/app/web/index.html");
27         window.set_html(kIndexHtml);
28         window.run();
29     } catch (const webview::exception &e) {
30         std::cerr << "webview 启动失败: " << e.what() << '\n';
31         return 1;
32     }
33
34     return 0;
35 }
```

```
1  #pragma once
2
3  #include "presenter.h"
4
5  #include "day5_2/person.h"
6
7  #include <webview/webview.h>
8
9  #include <string>
10
11 namespace day5_2::app {
12
13 // 控制器：把界面上的动作翻译成业务调用，再把业务结果翻译回界面能用的数据。
14 // 它认识 webview 和 JSON，但自己不含任何业务规则。
15 class Controller {
16 public:
17     Controller(webview::webview &window, PersonService &service, Presenter presenter);
18
19     // 把三个动作注册到页面上：window.getState / submitEntry / clearMessages
20     void bind();
21
22 private:
23     std::string handleState();
24     std::string handleSubmit(const std::string &argsJson);
25     std::string handleClear();
26
27     // 组装一次回包：状态文本用当前 m_statusText，列表统一走 Presenter。
28     std::string reply(bool ok, const char *level, std::string event, std::string detail,
29                     std::string errorKey);
30
31     webview::webview &m_window;
32     PersonService &m_service;
33     Presenter m_presenter;
34     std::string m_statusText;
35 };
36
37 } // namespace day5_2::app
```

```
1  #include "controller.h"
2
3  #include "protocol.h"
4
5  #include <iostream>
6  #include <utility>
7
8  namespace day5_2::app {
9
10 Controller::Controller(webview::webview &window, PersonService &service, Presenter presenter)
11     : m_window(window), m_service(service), m_presenter(std::move(presenter)),
12       m_statusText(m_presenter.idleStatus()) {}
13
14 void Controller::bind() {
15     m_window.bind("getState",
16         [this](const std::string &) -> std::string { return handleState(); });
17
18     m_window.bind("submitEntry", [this](const std::string &args) -> std::string {
19         return handleSubmit(args);
20     });
21
22     m_window.bind("clearMessages",
23         [this](const std::string &) -> std::string { return handleClear(); });
24 }
25
26 std::string Controller::handleState() {
27     m_statusText = m_presenter.idleStatus();
28     return reply(true, "info", "state", "", "");
29 }
30
31 std::string Controller::handleSubmit(const std::string &argsJson) {
32     SubmitArgs args;
33     if (!decodeSubmitArgs(argsJson, args)) {
34         const Error error{ErrorCode::invalid_input, "protocol.bad_args", {}};
35         m_statusText = m_presenter.failedStatus(error);
36         const std::string payload = reply(false, "error", "failed", m_presenter.messageFor(error),
37                                           error.key);
38         std::cout << "C++ 通知失败: " << m_statusText << '\n';
39         return payload;
40     }
41
42     const Result<Person> result = m_service.submit(args.name, args.age);
43
44     std::string payload;
45     if (result.ok()) {
46         m_statusText = m_presenter.submittedStatus(result.value);
47         payload = reply(true, "ok", "submitted", m_presenter.formatPerson(result.value), "");
48         std::cout << "C++ 通知成功: " << m_statusText << '\n';
49     } else {
50         m_statusText = m_presenter.failedStatus(result.error);
51         payload = reply(false, "error", "failed", m_presenter.messageFor(result.error),
52                       result.error.key);
53         std::cout << "C++ 通知失败: " << m_statusText << '\n';
54     }
55
56     // C++ -> JS: 用 dispatch 回到 UI 线程, 再主动把这次操作的事件推给页面。
57     m_window.dispatch([this, payload]() {
58         m_window.eval("window.__onBackendEvent && window.__onBackendEvent(" + payload + ")");
59     });
60
61     return payload;
62 }
63
64 std::string Controller::handleClear() {
65     m_service.clear();
66     m_statusText = m_presenter.clearedStatus();
67 }
```

```
67     return reply(true, "info", "cleared", "", "");
68 }
69
70 std::string Controller::reply(bool ok, const char *level, std::string event, std::string detail,
71                               std::string errorKey) {
72     Envelope envelope;
73     envelope.ok = ok;
74     envelope.level = level;
75     envelope.event = std::move(event);
76     envelope.status = m_statusText;
77     envelope.detail = std::move(detail);
78     envelope.error = std::move(errorKey);
79     envelope.messages = m_presenter.formatPeople(m_service.people());
80     return encode(envelope);
81 }
82
83 } // namespace day5_2::app
```

```
1  #pragma once
2
3  #include "day5_2/person.h"
4  #include "day5_2/result.h"
5
6  #include <string>
7  #include <vector>
8
9  namespace day5_2::app {
10
11  // 展示层：整个项目里唯一生成面向用户文本的地方。
12  //
13  // core 只给"事实"（成功的数据，或错误码 + key），这里负责把事实变成人话。
14  // 换语言、换图标、换成英文界面，都只改这一个类。
15  class Presenter {
16  public:
17      std::string idleStatus() const;
18      std::string clearedStatus() const;
19      std::string submittedStatus(const Person &person) const;
20      std::string failedStatus(const Error &error) const;
21
22      // 把一条记录格式化成列表里显示的样子。
23      std::string formatPerson(const Person &person) const;
24      std::vector<std::string> formatPeople(const std::vector<Person> &people) const;
25
26      // 错误码 / 错误 key -> 中文说明。
27      std::string messageFor(const Error &error) const;
28
29  private:
30      static std::string lookup(ErrorCode code, const std::string &key);
31  };
32
33  } // namespace day5_2::app
```

```
1  #include "presenter.h"
2
3  namespace day5_2::app {
4
5      namespace {
6
7          constexpr const char *kIdleText = "等待输入...";
8          constexpr const char *kClearedText = "已清空";
9
10     } // namespace
11
12     std::string Presenter::idleStatus() const {
13         return kIdleText;
14     }
15
16     std::string Presenter::clearedStatus() const {
17         return kClearedText;
18     }
19
20     std::string Presenter::formatPerson(const Person &person) const {
21         return person.name + " (" + std::to_string(person.age) + " 岁)";
22     }
23
24     std::vector<std::string> Presenter::formatPeople(const std::vector<Person> &people) const {
25         std::vector<std::string> lines;
26         lines.reserve(people.size());
27         for (const Person &person : people)
28             lines.push_back(formatPerson(person));
29         return lines;
30     }
31
32     std::string Presenter::submittedStatus(const Person &person) const {
33         return "■ 已添加: " + formatPerson(person);
34     }
35
36     std::string Presenter::messageFor(const Error &error) const {
37         return lookup(error.code, error.key);
38     }
39
40     std::string Presenter::failedStatus(const Error &error) const {
41         return "✕ " + messageFor(error);
42     }
43
44     std::string Presenter::lookup(ErrorCode code, const std::string &key) {
45         // 先按 key 精确匹配：每个业务域可以有自己更具体的说法。
46         if (key == "person.name.empty")
47             return "姓名不能为空";
48         if (key == "person.age.out_of_range")
49             return "年龄必须在 1-150 之间";
50         if (key == "protocol.bad_args")
51             return "前端传来的参数无法解析";
52
53         // 再按错误码兜底，保证任何 key 都有话说。
54         switch (code) {
55             case ErrorCode::invalid_input:
56                 return "输入不合法";
57             case ErrorCode::not_found:
58                 return "没有找到对应的记录";
59             case ErrorCode::conflict:
60                 return "操作冲突";
61             case ErrorCode::internal:
62                 return "内部错误";
63             case ErrorCode::none:
64                 break;
65         }
66         return "未知错误";
67     }
```

```
67 | }  
68 |  
69 | } // namespace day5_2::app
```

```
1  #pragma once
2
3  #include <string>
4  #include <vector>
5
6  namespace day5_2::app {
7
8  // 前端 -> 后端的参数。具名字段，比位置数组 [{0},{1}] 更耐改。
9  struct SubmitArgs {
10     std::string name;
11     int age = 0;
12 };
13
14 // 解析 submitEntry(name, age) 传来的参数数组。失败返回 false，不抛异常
15 // （异常从 webview 的 C 回调里逃出去是不安全的）。
16 bool decodeSubmitArgs(const std::string &argsJson, SubmitArgs &out);
17
18 // 线上协议信封：字段名就是前后端契约，改这里等于改协议。
19 //   level : "info" 中性 / "ok" 成功 / "error" 失败 — 界面据此决定样式，
20 //   不再靠嗅探 ❏x 这类文本前缀。
21 //   error : 失败时的机器可读 key，成功后为空。
22 struct Envelope {
23     bool ok = false;
24     std::string level;
25     std::string event;
26     std::string status;
27     std::string detail;
28     std::string error;
29     std::vector<std::string> messages;
30 };
31
32 std::string encode(const Envelope &envelope);
33
34 } // namespace day5_2::app
```

```
1  #include "protocol.h"
2
3  #include <nlohmann/json.hpp>
4
5  namespace day5_2::app {
6
7  bool decodeSubmitArgs(const std::string &argsJson, SubmitArgs &out) {
8      // 第三个参数 false = 解析失败返回 discarded 值而不是抛异常。
9      const nlohmann::json args = nlohmann::json::parse(argsJson, nullptr, false);
10     if (!args.is_array())
11         return false;
12
13     if (args.size() > 0 && args[0].is_string())
14         out.name = args[0].get<std::string>();
15     if (args.size() > 1 && args[1].is_number())
16         out.age = args[1].get<int>();
17     return true;
18 }
19
20 std::string encode(const Envelope &envelope) {
21     // ordered_json: 字段顺序与写入顺序一致，读日志更直观。
22     return nlohmann::ordered_json{{"ok", envelope.ok},
23                                     {"level", envelope.level},
24                                     {"event", envelope.event},
25                                     {"status", envelope.status},
26                                     {"detail", envelope.detail},
27                                     {"error", envelope.error},
28                                     {"messages", envelope.messages}}
29         .dump();
30 }
31
32 } // namespace day5_2::app
```

```
1 <!DOCTYPE html>
2 <html lang="zh-CN">
3 <head>
4 <meta charset="utf-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1">
6 <title>Mini 数据流转示例</title>
7 <style>
8   * { box-sizing: border-box; }
9   html, body { height: 100%; margin: 0; }
10  body {
11    font-family: system-ui, "Noto Sans CJK SC", "Source Han Sans SC", sans-serif;
12    font-size: 14px;
13    background: #f4f5f7;
14    color: #1f2328;
15    user-select: none;
16  }
17  .app {
18    display: flex;
19    flex-direction: column;
20    gap: 12px;
21    height: 100%;
22    padding: 16px;
23  }
24  .field {
25    height: 34px;
26    padding: 0 10px;
27    border: 1px solid #c9ced6;
28    border-radius: 6px;
29    background: #fff;
30    font: inherit;
31    color: inherit;
32    outline: none;
33  }
34  .field:focus {
35    border-color: #3b82f6;
36    box-shadow: 0 0 0 3px rgba(59, 130, 246, 0.15);
37  }
38  .row { display: flex; gap: 10px; }
39  .btn {
40    flex: 1;
41    height: 34px;
42    border: 1px solid #c9ced6;
43    border-radius: 6px;
44    background: #fff;
45    font: inherit;
46    color: inherit;
47    cursor: pointer;
48  }
49  .btn:hover { background: #f0f2f5; }
50  .btn.primary {
51    border-color: #3b82f6;
52    background: #3b82f6;
53    color: #fff;
54  }
55  .btn.primary:hover { background: #2f74e0; }
56  .status {
57    margin: 0;
58    min-height: 20px;
59    color: #8b949e;
60    word-break: break-word;
61  }
62  .status.ok { color: #15803d; }
63  .status.err { color: #dc2626; }
64  .list {
65    flex: 1;
66    margin: 0;
```

```

67     padding: 0;
68     overflow-y: auto;
69     list-style: none;
70     border: 1px solid #e3e6ea;
71     border-radius: 6px;
72     background: #fff;
73 }
74 .item { padding: 10px 12px; border-bottom: 1px solid #f0f2f5; }
75 .item:last-child { border-bottom: 0; }
76 .item.alt { background: #fafbfc; }
77 .list.empty { padding: 10px 12px; color: #9aa4b0; }
78 </style>
79 </head>
80 <body>
81   <main class="app">
82     <input id="name-field" class="field" type="text" placeholder="姓名" autocomplete="off">
83     <input id="age-field" class="field" type="number" min="1" max="150" value="20">
84     <div class="row">
85       <button id="submit-button" class="btn primary" type="button">提交</button>
86       <button id="clear-button" class="btn" type="button">清空</button>
87     </div>
88     <p id="status-text" class="status">等待输入...</p>
89     <ul id="message-list" class="list"></ul>
90   </main>
91
92   <script>
93     "use strict";
94
95     var nameField = document.getElementById("name-field");
96     var ageField = document.getElementById("age-field");
97     var submitButton = document.getElementById("submit-button");
98     var clearButton = document.getElementById("clear-button");
99     var statusText = document.getElementById("status-text");
100    var messageList = document.getElementById("message-list");
101
102    // 样式只认协议里的 level 字段 (info / ok / error),
103    // 不去嗅探 ✖ 这类文本前缀——文案可以在后端随便改, 界面不受影响。
104    var STATUS_CLASS = { info: "status", ok: "status ok", error: "status err" };
105
106    function renderStatus(response) {
107      statusText.textContent = response.status || "";
108      statusText.className = STATUS_CLASS[response.level] || "status";
109    }
110
111    function renderMessages(messages) {
112      messageList.textContent = "";
113      var items = messages || [];
114      if (items.length === 0) {
115        var hint = document.createElement("li");
116        hint.className = "empty";
117        hint.textContent = " (暂无数据) ";
118        messageList.appendChild(hint);
119        return;
120      }
121      items.forEach(function (item, index) {
122        var li = document.createElement("li");
123        li.className = "item" + (index % 2 ? " alt" : "");
124        li.textContent = (index + 1) + ". " + item;
125        messageList.appendChild(li);
126      });
127    }
128
129    function applyResponse(response) {
130      if (!response) return;
131      renderStatus(response);
132      renderMessages(response.messages);
133    }
134

```

```

135 // C++ 通过 webview 的 eval() 主动调用这里，把一次操作的事件推给页面。
136 // error 字段是机器可读的失败原因，需要精确处理某个错误时用它。
137 window.__onBackendEvent = function (event) {
138     console.log("C++ 通知:", event.event, "->", event.detail, "| level =", event.level, "| error =",
        event.error);
139 };
140
141 // webview 的绑定是在页面脚本之前注入的，这里再等一等，保证 window.getState 已经就绪。
142 function whenBackendReady(callback) {
143     var attempts = 0;
144     var timer = setInterval(function () {
145         if (typeof window.getState === "function") {
146             clearInterval(timer);
147             callback();
148         } else if (++attempts > 150) {
149             clearInterval(timer);
150             renderStatus({ status: "× 未能连接到 C++ 后端", level: "error" });
151         }
152     }, 20);
153 }
154
155 whenBackendReady(function () {
156     window.getState().then(applyResponse).catch(function (error) {
157         renderStatus({ status: "× 初始化失败: " + error, level: "error" });
158     });
159
160     submitButton.addEventListener("click", function () {
161         var name = nameField.value;
162         var age = Number(ageField.value);
163         window.submitEntry(name, age).then(function (response) {
164             applyResponse(response);
165             nameField.value = "";
166             nameField.focus();
167         });
168     });
169
170     clearButton.addEventListener("click", function () {
171         window.clearMessages().then(applyResponse);
172     });
173 });
174 </script>
175 </body>
176 </html>

```

```
1 | # tests: 脱离界面测试业务逻辑。
2 | # core 是零 UI 依赖的静态库，所以这里不需要窗口、不需要 GTK，直接跑。
3 | add_executable(person_test person_test.cpp)
4 | target_link_libraries(person_test PRIVATE day5_core)
5 | target_compile_options(person_test PRIVATE -Wall -Wextra -Wpedantic)
6 |
7 | add_test(NAME person_test COMMAND person_test)
```

```
1 // 业务测试：不启动界面，直接调用 core 层的函数。
2 // 没有引入测试框架——assert 风格的检查就够小项目用了，少一个依赖少一件事。
3 #include "day5_2/person.h"
4
5 #include <iostream>
6 #include <string>
7
8 namespace {
9
10 int g_failures = 0;
11
12 void check(bool condition, const std::string &what) {
13     if (condition) {
14         std::cout << "ok   " << what << '\n';
15     } else {
16         std::cout << "FAIL " << what << '\n';
17         ++g_failures;
18     }
19 }
20
21 void empty_name_is_rejected() {
22     day5_2::PersonService service;
23     const auto result = service.submit("   ", 20);
24
25     check(!result.ok(), "只有空白的姓名被拒绝");
26     check(result.error.key == "person.name.empty", "失败原因是 person.name.empty");
27     check(result.error.code == day5_2::ErrorCode::invalid_input, "失败分类是 invalid_input");
28     check(service.people().empty(), "被拒绝的输入不会写进列表");
29 }
30
31 void age_out_of_range_is_rejected() {
32     day5_2::PersonService service;
33
34     check(!service.submit("张三", 0).ok(), "年龄 0 被拒绝");
35     check(!service.submit("张三", 151).ok(), "年龄 151 被拒绝");
36     check(service.people().empty(), "越界的输入一条都没写进去");
37 }
38
39 void age_boundaries_are_accepted() {
40     day5_2::PersonService service;
41
42     check(service.submit("最小", 1).ok(), "年龄 1 通过");
43     check(service.submit("最大", 150).ok(), "年龄 150 通过");
44     check(service.people().size() == 2, "两条边界记录都写进去了");
45 }
46
47 void name_is_trimmed_and_stored_as_data() {
48     day5_2::PersonService service;
49     const auto result = service.submit("   张三   ", 20);
50
51     check(result.ok(), "合法输入通过");
52     check(result.value.name == "张三", "姓名去掉了首尾空白");
53     check(result.value.age == 20, "年龄原样保留");
54     check(service.people().size() == 1, "列表里有一条记录");
55     check(service.people()[0].name == "张三", "存的是数据本身，不是格式化后的字符串");
56 }
57
58 void clear_emptyies_the_list() {
59     day5_2::PersonService service;
60     service.submit("张三", 20);
61     service.clear();
62
63     check(service.people().empty(), "清空之后列表为空");
64 }
65
66 } // namespace
```

```
67
68 int main() {
69     empty_name_is_rejected();
70     age_out_of_range_is_rejected();
71     age_boundaries_are_accepted();
72     name_is_trimmed_and_stored_as_data();
73     clear_emptyies_the_list();
74
75     if (g_failures == 0) {
76         std::cout << "\n全部通过\n";
77         return 0;
78     }
79     std::cout << "\n失败 " << g_failures << " 项\n";
80     return 1;
81 }
```